

JIT for Dynamic Programming Languages Considered Easy

Taine Zhao
University of Tsukuba

Lambda Days & TFP, 2021

- ▶ Background
- ▶ Call-site Specialisation: IIS and PCS
- ▶ Assumptions and Optimisations
- ▶ Use for Partial Evaluation
- ▶ Pros and Cons

Dynamic programming languages are getting popular...

but... JIT?

Issues:

- ▶ non-technical: human resources, money...
- ▶ technical: compatibility(C extensions, etc.), complexity, time...

A working approach applied to CPython:

- ▶ implemented within 1000 lines
- ▶ non-invasive changes to language implementation
- ▶ compatibility
- ▶ extensibility

Call-site specialisation is common:

$$f(arg_1 \cdots arg_n)$$

By partly knowing the information of the callable f , and its arguments $arg_1 \cdots arg_n$, we may be able to

1. choose the appropriate implementation of f
2. eliminate runtime type checks
3. infer the representation of the return

```
1 import diojit as jit
2
3 def append3(xs, x):
4     xs.append(x)
5     xs.append(x)
6     xs.append(x)
7
8 jit_append3 = jit.specialise(
9     append3, jit.oftype(list), jit.Top)
10 jit_append3(...) # performance gain: >100%
```

An *immediately invoked specialisation*(IIS) is the specialisation performed on a callable exactly when executing the code. JIT compilation is triggered here, which is time-consuming.

IISs are slow, but chances are there to make programs faster.

```
1 import diojit as jit
2
3 def append3(xs, x):
4     →xs.append(x)
5     →xs.append(x)
6     →xs.append(x)
7
8 jit_append3 = jit.specialise(
9     append3, jit.oftype(list), jit.Top)
10 # performance gain: >100%
11 jit_append3(...)
```

A *pre-call specialisation* (PCS) happens inside an IIS.

It is performed on a callable ****before**** executing the code.

```
1 def append3(xs, x):  
2     xs.append(x)  
3  
4 jit_append3 = jit.specialise(  
5     append3, jit.oftype(list), jit.Top)  
6 jit_append3(...)
```

when $\sigma = \{xs : \text{list}, x : \top\}$

`xs.append(x)`

`xs.append(x)`

Specialisation rules used at analysis time:

When `xs` is a `list`

case `xs.append(a1)` $\Rightarrow$ *CListAppend*(`xs`, `a1`) : `unit`

case `xs.append( $\bar{a}$ )` $\Rightarrow$ (fall back to CPython) : $\top$

where *CListAppend* is a specialised implementation that

1. avoids type-checking the arguments
2. eliminates dynamic method lookup
3. guarantees the return value is `None`, which holds the type `unit`.


```
1 class Node:
2     def __init__(self, n, val):
3         self.next = n
4         self.val = val
5
6 def sum_chain(n):
7     a = 0
8     while n is not None:
9         a += n.val
10        n = n.next
11    return a
12
13 n0 = Node(None, 0)
14 ni+1 = Node(ni, i)
15 sum_chain(n100)
```

Running it a million times costs **8.75s**. It's slow. The performance is poor because...

6.23s / million. 40% speed up.

```

1  class Node:
2      def __init__(self, n, val):
3          self.next = n
4          self.val = val
5
6  def sum_chain(n):
7      a = 0
8      while n is not None:
9          a += n.val
10         n = n.next
11     return a
12
13 jit_func = jit.specialise(sum_chain, jit.oftype(Node))
14 jit_func(n_N)

```

PC	Store	Jump
7	{n : Node}	8
8	{a : int = 0, n : Node}	9
9	{a : int = 0, n : Node}	10
10	{a : T, n : Node}	8
8	{a : T, n : T}	9 or 11
9	{a : T, n : T}	10
10	{a : T, n : T}	8
8	{a : T, n : T}	terminate

```

1  unit = type(None)
2  @jit.eagerjit
3  class Node:
4      next: Union[Node, unit]
5      val: int
6      def __init__(self, n, val):
7          self.next = n
8          self.val = val
9
10 def sum_chain(n):
11     a = 0
12     while n is not None:
13         a += n.val
14         n = n.next
15     return a

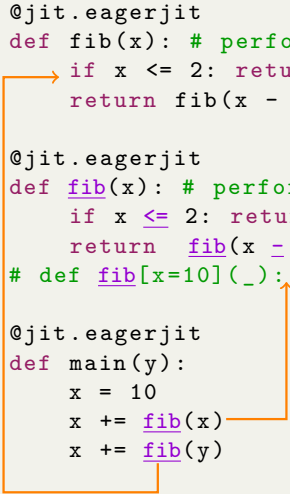
```

PC	Store	Jump
11	{n : Node}	12
12	{a : int = 0, n : Node}	13
13	{a : int = 0, n : Node}	14
14	{a : int, n : Node}	12
12	{a : int, n : unit Node}	U_1 or U_2
U_1	{a : int, n : Node}	13
13	{a : int, n : Node}	14
14	{a : int, n : Node}	terminate
U_2	{a : int, n = None}	12
12	{a : int, n = None}	15

3.57s / million, 140% performance gain.

func is a variant of `func` that tries computations at analysis time.

```
1 @jit.eagerjit
2 def fib(x): # performance gain: >600%
3     if x <= 2: return 1
4     return fib(x - 1) + fib(x - 2)
5
6 @jit.eagerjit
7 def fib(x): # performance gain: >35000%
8     if x <= 2: return 1
9     return fib(x - 1) + fib(x - 2)
10 # def fib[x=10](_): return 55
11
12 @jit.eagerjit
13 def main(y):
14     x = 10
15     x += fib(x)
16     x += fib(y)
17
18 jit_main = jit.specialise(main, jit.oftype(int))
19 jit_main()
```



```
1 from jit import register, Judge, AbsVal, S, CallSpec
2 import operator # python operator implementation
3 @register(+, method="__call__")
4 def call_const_add(self: Judge, *args: AbsVal):
5     if len(args) != 2:
6         # no specialisation
7         return NotImplemented
8     [a, b] = args
9     if a.is_static() and b.is_static():
10        # .base get the static value if applicable
11        const = a.base + b.base
12        type_const = type(const)
13        ret_types = (S(type_const), )
14        return CallSpec(const, const, ret_types)
15
16    # roll back to ordinary 'a + b' specialisation
17    static_fn = S(operator.__add__)
18    return self.spec(static_fn, "__call__", args)
```

Item	PY38	JIT PY38	PY39	JIT PY39
brainf**k	265.74	134.23	244.50	140.34
append3	23.94	10.70	22.29	11.21
DNA READ	16.96	14.82	15.03	14.38
fib(15)	11.63	1.54	10.41	1.51
hypot	6.19	3.87	6.53	4.29
selectsort	46.95	33.88	38.71	29.49

Pros:

- ▶ minimal: core within 1000 lines
- ▶ non-invasive: implemented as a 3rd-party library
- ▶ compatible: encouraging C extensions
- ▶ extensible: defining new specialisation rules
- ▶ functional: benefiting from immutability and small functions

Cons:

- ▶ issues with non-generics: when `xs` is analysed as a `list`, we cannot analyse the type of `xs[0]`
- ▶ issues with dynamic scoping: deciding which object a global variable is referencing is a must
- ▶ generated code size is huge